ONLINE SCHEMA MIGRATIONS • PRODUCTION POST-MORTEM

How an ALTER TABLE Caused a Massive GitHub Outage: gh-ost Breakdown

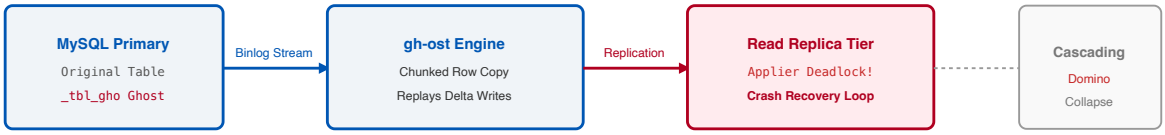
Dissection of gh-ost Binlog Replay Deadlocks, Crash Recovery Loops, and Cascading Replica Collapses

EXECUTIVE SUMMARY & OUTAGE SCOPE

On November 27, 2020, an online schema migration using `gh-ost` on a massive MySQL table triggered unhandled deadlocks inside the replication SQL applier on read replicas. This plunged read nodes into sequential crash-recovery states. As each replica crashed, ProxySQL load balancers redirected 100% of read traffic onto the surviving nodes, triggering an immediate, fatal cascading failure across GitHub's primary read tier.

1. THE SCALE DILEMMA: WHY ALTER TABLE IS DANGEROUS

At enterprise scale, altering a table with billions of rows using native DDL locks tables, burns I/O bandwidth, and accumulates catastrophic replication lag. GitHub developed `gh-ost` to perform asynchronous, triggerless migrations:



2. ROOT CAUSE MECHANISM: THE DEADLOCK AND CRASH LOOP

During active schema migrations on busy tables, continuous client inserts, updates, and deletes generate thousands of binlog events. When `gh-ost` copies bulk row batches simultaneously, row-level locks conflict with incoming DML:

InnoDB Replication Deadlock

The single-threaded SQL applier thread on replicas hit a deadlock exception during multi-statement transaction application. Rather than gracefully retrying, the replica marked the binary log position corrupted and terminated `mysqld`.

Crash Recovery Thrashing

Upon auto-restart, InnoDB enters crash recovery mode: replaying redo logs and rolling back incomplete undo transactions. This process consumes 100% of disk I/O for 10–20 minutes before the database can serve incoming reads.

3. MATHEMATICAL MODEL OF CASCADING FAILURE COLLAPSE

When multiple replicas share a total ingress workload, losing a single replica shifts disproportionate load to survivors. If survivors lack buffer capacity, a domino collapse is mathematically inevitable:

```
Initial Cluster State:
  Total read ingress: R = 3,000 req/sec across N = 3 replicas
  Baseline load per node: L_0 = 3,000 / 3 = 1,000 req/sec
  Provisioned maximum capacity: C_max = 1,300 req/sec (30% buffer)

Step 1: Replica #1 crashes due to replication deadlock.
  Remaining healthy nodes: N' = 2
  Load per surviving node: L_1 = 3,000 / 2 = 1,500 req/sec!
  Overload ratio: 1,500 / 1,300 = 1.15x (Exceeds capacity by 15%)

Step 2: Replica #2 exhausts connection pool and crashes.
  Remaining healthy nodes: N'' = 1
  Load on survivor: L_2 = 3,000 / 1 = 3,000 req/sec!
  Overload ratio: 3,000 / 1,300 = 2.30x -> Crashes in under 15 seconds!
```

4. OUTAGE INCIDENT CHRONOLOGY

Timestamp	Component	Observed Anomaly & Impact	Severity Tier
T + 00:00	Migration Engine	`gh-ost` starts copying chunks on multi-terabyte primary table.	Routine Normal Run
T + 00:15	Read Replica #1	SQL thread hits deadlock during binlog replay; daemon aborts.	Degraded 1 Node Down
T + 00:18	ProxySQL Proxy	Health check marks Replica #1 dead; splits 100% traffic across #2 & #3.	1.5x Load Overcapacity
T + 00:22	Read Replica #2	High latency and connection pool exhaustion trigger crash.	Critical 2 Nodes Down
T + 00:25	Read Replica #3	Receives 3,000 req/sec; CPU pins at 100%; mysqld killed by OOM.	Total 0 Nodes Active
T + 00:30	GitHub Ingress	Sitewide 500 errors across Actions, PRs, Issues, and Webhooks.	Outage Sitewide Outage

5. DEEP ARCHITECTURAL INSIGHTS

- **Lack of Read Replica Segregation:** Customer-facing read traffic and internal batch transformations (gh-ost) shared the same replica pool, violating failure domain isolation boundaries.
- **Absence of Proxy Load Shedding:** ProxySQL aggressively pushed 100% of inbound client queries to surviving nodes rather than capping maximum requests per backend and shedding excess at the edge.

6. ARCHITECTURE SOLUTIONS: DEDICATED POOLS & INTELLIGENT THROTTLING

To prevent schema migrations from ever threatening production read paths, GitHub instituted structural changes to their database topology and migration policies:

1. Dedicated Migration Replicas

Isolated replication nodes designated strictly for `gh-ost` binlog streaming and chunk inspection. Production user queries never touch migration replicas.

2. Segregated Read Pools

Customer read traffic is completely segregated from internal batch analytics, reporting dashboards, and backup snapshots, preventing cross-tenant exhaustion.

3. Dynamic Heartbeat Throttling

`gh-ost` monitors replica lag via high-frequency heartbeats. If lag exceeds 1000ms or running threads exceed 80, backfill throttles instantly.

7. HARDENED GH-OST DEPLOYMENT INVARIANTS

```
# Production-hardened gh-ost execution pattern:
gh-ost   --max-lag-millis=1000   --throttle-control-replicas="db-rep-migration-01,db-rep-migration-02"   --chunk-size=500   --max-load="Threads_running=50"   --critical-load="Threads_running=100"   --serve-socket-file="/tmp/gh-ost.sock"   --alter="ADD COLUMN reviewer_state VARCHAR(32)"   --database="github_production"   --table="pull_requests"   --execute
```

8. PRODUCTION ENGINEERING CHECKLIST

Core Principles for High-Availability Database Clusters

1. **Physically isolate migration traffic:** Never execute backfills against customer-facing read replicas.
2. **Implement proxy-level rate caps:** Configure database load balancers (ProxySQL) with max connection caps per node and drop surplus traffic gracefully (HTTP 429).
3. **Monitor replication heartbeat lag:** Pause asynchronous transformations automatically when replica lag climbs above sub-second thresholds.
4. **Over-provision read clusters:** Size replica pools so losing $N-1$ nodes can be absorbed safely under standard traffic volumes.
5. **Conduct regular disaster simulations:** Simulate deadlocks and sudden replica node crashes under production load during scheduled drills.